

Extracción del FIN TRACE y cálculo del MONTO

Explicación línea por línea del código (versión por rango de trace)

CLIENTE **Fedecrédito**

PROVEEDOR **E2ASolutions**

ALCANCE **Cajeros BNA**

Cómo explicarlo — versión simple

En una frase: "El robot suma todos los depósitos que entraron entre el corte anterior y el corte de hoy, y compara ese total contra lo que se contó físicamente."

Con una comparación fácil:

- El **trace** es como un **número de ticket** que la máquina le pone a cada depósito, uno tras otro (1, 2, 3, 4...).
- Un arqueo es **un tramo de esos tickets**: desde donde quedó el arqueo pasado hasta donde corta el de hoy.
- **INICIO** = dónde terminó el arqueo anterior (guardado en el maestro, así se encadena sin huecos ni repetidos).
- **FIN** = dónde corta el de hoy. Cuando la máquina abre un lote nuevo imprime una marca ("transacción no registrada") con su número de ticket; el robot toma la primera marca de la fecha del arqueo → ese es el fin.
- **MONTO** = el robot suma **solo las transacciones cuyo ticket cae dentro de ese tramo**. Eso es lo que la gente depositó en el periodo.
- Después compara ese monto (**lo registrado**) contra el **total contado físicamente**. Si no coincide → **faltante o sobrante**.

Dos detalles por si repreguntan

- **"¿Por qué el contador a veces parece ir para atrás?"** → El número de ticket solo llega a 9999 y vuelve a 0 (como el odómetro de un carro que da la vuelta). El robot lo detecta para no perder los depósitos de después de la vuelta.
- **"¿Y las transacciones anuladas?"** → No se cuentan; solo se suman las normales (los reversos se descartan).

DETALLE TÉCNICO

Idea general. El arqueo suma los depósitos de un **rango de números de secuencial (trace)**: desde donde cerró el arqueo anterior (**INICIO TRACE**, tomado del maestro) hasta donde corta este (**FIN TRACE**, extraído del reporte). Todo lo que caiga en ese rango, sumado, es el **MONTO** del periodo.

DATOS REALES USADOS COMO EJEMPLO

LOTE # 000427 -----
TRANS.NO REGISTRADA -35- (\$)
411201XXXXXX2188 4807
2026/06/20 \$ 0.00

Al partir el texto que sigue a TRANS.NO REGISTRADA por espacios/saltos de línea:

Índice	Valor	Qué es
TOK[0]	" "	vacío (hay un espacio tras la etiqueta)
TOK[1]	-35- (\$)	un código
TOK[2]	411201XXXXXX2188	la tarjeta
TOK[3]	4807	el trace (secuencial) — lo que buscamos
TOK[4]	2026/06/20	la fecha

Bloque 1 — FIN TRACE

```
Text.SplitText... Text: FechaF CustomDelimiter: '/' Result=> pf
```

Parte la fecha de arqueo **FechaF** (texto "20/06/2026") por **/**. Deja la lista **pf**: pf[0]="20" (día), pf[1]="06" (mes), pf[2]="2026" (año). *PAD indexa desde 0.*

```
SET pfKey T0 pf[2] + pf[1] + pf[0]
```

El **+** entre textos **concatena** (no suma): "2026"+"06"+"20" = **"20260620"**. Reordena a **AAAAMMDD** para comparar fechas **como texto**: solo en ese orden la comparación alfabética (izquierda a derecha) equivale al orden cronológico real. Ej.: 30/05 vs 02/06 → "20260530" < "20260602" (correcto); en cambio "30052026" > "02062026" daría al revés.

```
SET loteAnterior T0 ''
```

Inicializa vacío el "último lote visto", para detectar cuándo cambia de lote.

```
SET finTrace T0 ''
```

Inicializa vacío el resultado (todavía no se encontró el corte).

```
LOOP FOREACH CurrentReg IN repuestaTexto
```

Recorre cada bloque de la respuesta AT33.

```
IF Contains(CurrentReg.datos, 'LOTE #', False) THEN
```

Solo entra si el bloque contiene **LOTE #** (es un comprobante de lote); el resto se ignora.

```
Text.SplitText... Text: CurrentReg.datos CustomDelimiter: 'LOTE #' Result=> PL
```

Parte el bloque por **LOTE #**. **PL[1]** = lo que viene después (empieza con un espacio y luego el número: " 000427 - ---").

```
Text.GetSubtext... Text: PL[1] CharacterPosition: 1 NumberOfChars: 6 Subtext=> loteActual
```

De PL[1] toma 6 caracteres desde la posición 1 (salta el espacio del índice 0) → **loteActual = "000427"**. Por eso 1 (saltar espacio) y 6 (dígitos del lote).

```
Text.SplitText... Text: CurrentReg.datos CustomDelimiter: 'TRANS.NO REGISTRADA' Result=> PT
```

Parte el bloque por la etiqueta **TRANS.NO REGISTRADA**. **PT[1]** = el texto que sigue a esa etiqueta.

```
Text.SplitText... Text: PT[1] CustomDelimiter: '\s+' IsRegex: True Result=> TOK
```

Parte ese texto por espacios/saltos de línea (ver tabla arriba): TOK[0]="", TOK[1]="-35-\$", TOK[2]=tarjeta, TOK[3]=trace...

```
SET traceActual T0 TOK[3]
```

Toma el **4.º pedazo** = el número de trace de esa transacción de corte ("**4807**").

```
Text.SplitText... Text: CurrentReg.fechaCaptura CustomDelimiter: '/' Result=> pc
```

Parte la fecha de captura del bloque por **/**: pc[0] día, pc[1] mes, pc[2] año.

```
SET fechaKey T0 pc[2] + pc[1] + pc[0]
```

La deja en **AAAAMMDD** (igual que pfKey) para poder compararlas como texto.

```
IF loteActual <> loteAnterior THEN
```

Solo actúa cuando el lote **cambió** respecto al anterior: nos interesa el **borde** donde arranca un lote nuevo, no cada línea del mismo lote.

```
IF fechaKey >= pfKey THEN
```

Y solo si ese cambio de lote es de la **fecha de arqueo o posterior** (ignora cortes previos al periodo).

```
IF IsEmpty(finTrace) THEN
```

Y solo si aún **no** se guardó un finTrace → así se queda con el **primero** que cumple todo.

```
SET finTrace T0 traceActual
```

Guarda ese trace como **FIN TRACE**.

```
END → cierra IF IsEmpty(finTrace)
```

```
END → cierra IF fechaKey >= pfKey
```

```
SET loteAnterior T0 loteActual
```

Actualiza el "último lote visto" (dentro del IF de cambio de lote), para detectar el próximo cambio en la siguiente vuelta.

```
END → cierra IF loteActual <> loteAnterior
```

```
END → cierra IF Contains(... 'LOTE #' ...)
```

```
END → cierra LOOP FOREACH
```

Resultado: finTrace = el trace de la primera TRANS.NO REGISTRADA del **primer lote nuevo en o después de la fecha de arqueo**.

Bloque 2 — MONTO

Antes de este loop ya se definieron: iTn = inicioTrace como número · fTn = finTrace como número · $esWrap$ = True si $iTn > fTn$ (el rango cruzó el 9999 → 0).

```
LOOP FOREACH CurrentItem IN RespuestaA030.listaA030
```

Recorre cada transacción del A030.

```
IF CurrentItem.normaloReverso = '0' THEN
```

Solo procesa las **normales** (0); descarta los **reversos** para no ensuciar la suma.

```
Text.ToNumber Text: CurrentItem.secuencial Number=> secN
```

Convierte el secuencial (trace) de esa transacción, de texto a número, en **secN**.

```
SET enRango T0 False
```

Asume que **no** está en el rango, hasta probar lo contrario.

```
IF IsEmpty(finTrace) THEN
```

Caso de **respaldo**: si nunca se encontró finTrace...

```
IF secN >= iTn THEN → SET enRango T0 True → END
```

...entra todo lo que tenga secuencial desde el inicio en adelante (rango abierto).

```
ELSE (sí hay finTrace)
```

Caso normal, con corte definido:

```
IF esWrap THEN
```

Si el rango cruzó el 9999 → 0 (wrap del contador de traces)...

```
IF secN >= iTn THEN → enRango = True → END
```

...entra la **parte alta** (desde el inicio hasta 9999).

```
IF secN < fTn THEN → enRango = True → END
```

...y también la **parte baja** (desde 0 hasta antes del fin). Basta cumplir **una** de las dos ("o").

```
ELSE (sin wrap, rango normal)
```

Rango continuo, sin vuelta:

```
IF secN >= iTn THEN / IF secN < fTn THEN → enRango = True
```

Entra si $iTn \leq secN < fTn$. El **iTn** entra (es el primer trace de este periodo); el **fTn** no entra (es el corte del siguiente periodo).

EJEMPLO DE WRAP — INICIO 9995, FIN 0005

```
parte alta (secN >= 9995): 9995 9996 9997 9998 9999
parte baja (secN < 0005): 0000 0001 0002 0003 0004
el 0005 (fin) queda FUERA.
```

END → cierra IF esWrap / su ELSE

END → cierra IF IsEmpty(finTrace) / su ELSE

```
IF enRango THEN
```

Si la transacción quedó dentro del rango...

```
SET montoTrace TO montoTrace + CurrentItem.valor
```

...suma su **valor** al acumulado **montoTrace**.

```
END → cierra IF enRango
```

```
END → cierra IF normaloReverso = '0'
```

```
END → cierra LOOP FOREACH
```

Resultado: MONTO = suma de los va lor del A030 (solo normales) cuyo secuencial está en **[inicioTrace, finTrace)**, contemplando el wrap del contador 0-9999.